@ codedsoul_05

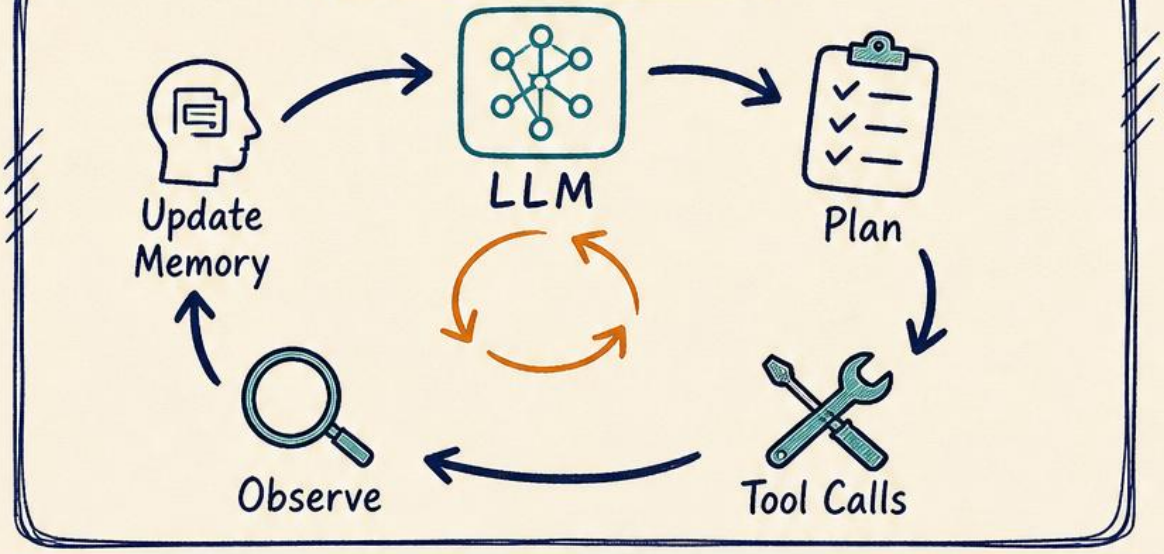
AGENTS v1.0

AGENTIC AI

HANDWRITTEN HANDBOOK

— BEGINNER TO PRODUCTION —

Building LLM Agents that Plan, Use Tools, Remember and Finish Jobs



LOOP • TOOLS • MEMORY • MULTI-AGENT • GUARDRAILS • EVAL

WHAT'S INSIDE

1. Agentic AI Fundamentals
2. Agent Patterns
3. Prompting for Agents
4. Tool Use and Function Calling
5. Memory and Context
6. RAG Memory
7. Planning Loops
8. Multi-Agent Collaboration
9. Guardrails and Safety
10. Evaluation and Observability
12. Cost and Latency
12. Real-World Examples

BY THE NUMBERS

- 12 Modules
- 30+ Tools
- 10+ Memory Types
- Multi-Agent Ready
- Safety First
- Measure Everything

TOPICS COVERED

- [x] Agentic Principles
- [x] Planning and Reasoning
- [x] Tool Use and Automation
- [x] Memory and Context
- [x] Retrieval and RAG
- [x] Loops and Feedback
- [x] Multi-Agent Systems
- [x] Safety and Guardrails
- [x] Evaluation and Metrics
- [x] Performance and Cost
- [x] Real-World Applications

★ PLAN. ACT. OBSERVE. REPEAT — WITH BRAKES. ★

01 — WHAT IS AGENTIC AI?

Agentic AI = LLM + tools + memory + a control loop toward a goal.

AUTONOMY (INCREASES →)



Script / RPA (rule-based)	Chatbot (single turn)	Tool-calling assistant (uses tools)	Agentic workflow (multi-step)	Multi-agent system (collaborative)

CAPABILITIES

- [x] Decompose goals
- [x] Choose tools
- [x] Maintain state (memory)
- [x] Retry / recover from failures
- [x] Ask clarifying questions
- [x] Stop when done (halt condition)

vs CHATGPT DESIGN

ChatGPT handbook = inference serving at scale.

Agentic AI = decision / control plane on top of models.

WHEN TO USE

- Research & information synthesis
- Coding agents (plan → implement → test)
- Ops runbooks & incident resolution
- Customer workflows needing systems of record

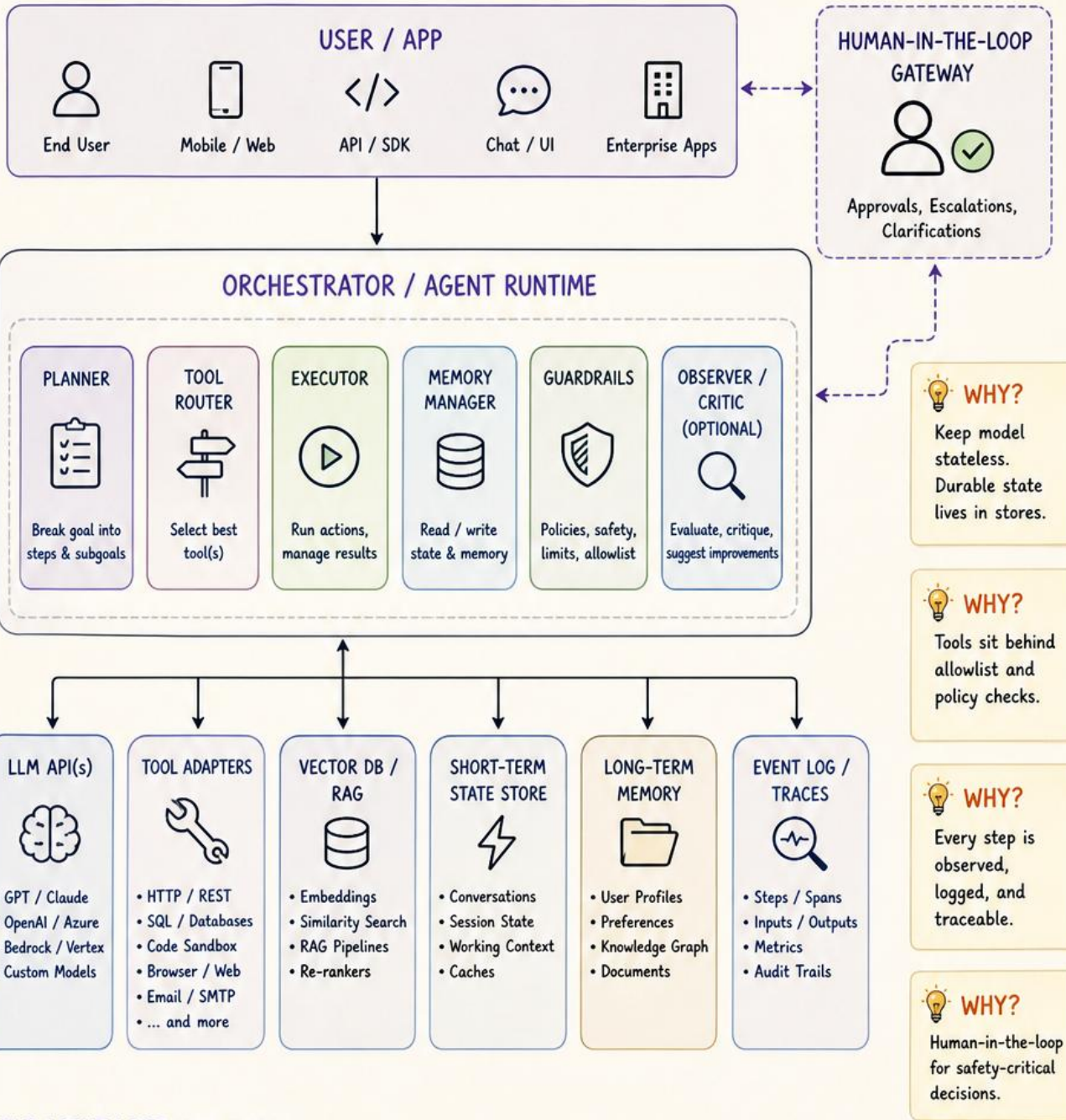
WHEN NOT TO USE

- Pure classify / extract tasks
- Ultra-low latency, single-shot needs
- Highly regulated workflows without human gates

TIP: In interviews, define the goal, tools, stop conditions, and failure policy before drawing boxes.

★ Goal-directed loop beats one-shot prompt.

02 — High-Level Architecture



WHY?
Keep model stateless. Durable state lives in stores.

WHY?
Tools sit behind allowlist and policy checks.

WHY?
Every step is observed, logged, and traceable.

WHY?
Human-in-the-loop for safety-critical decisions.

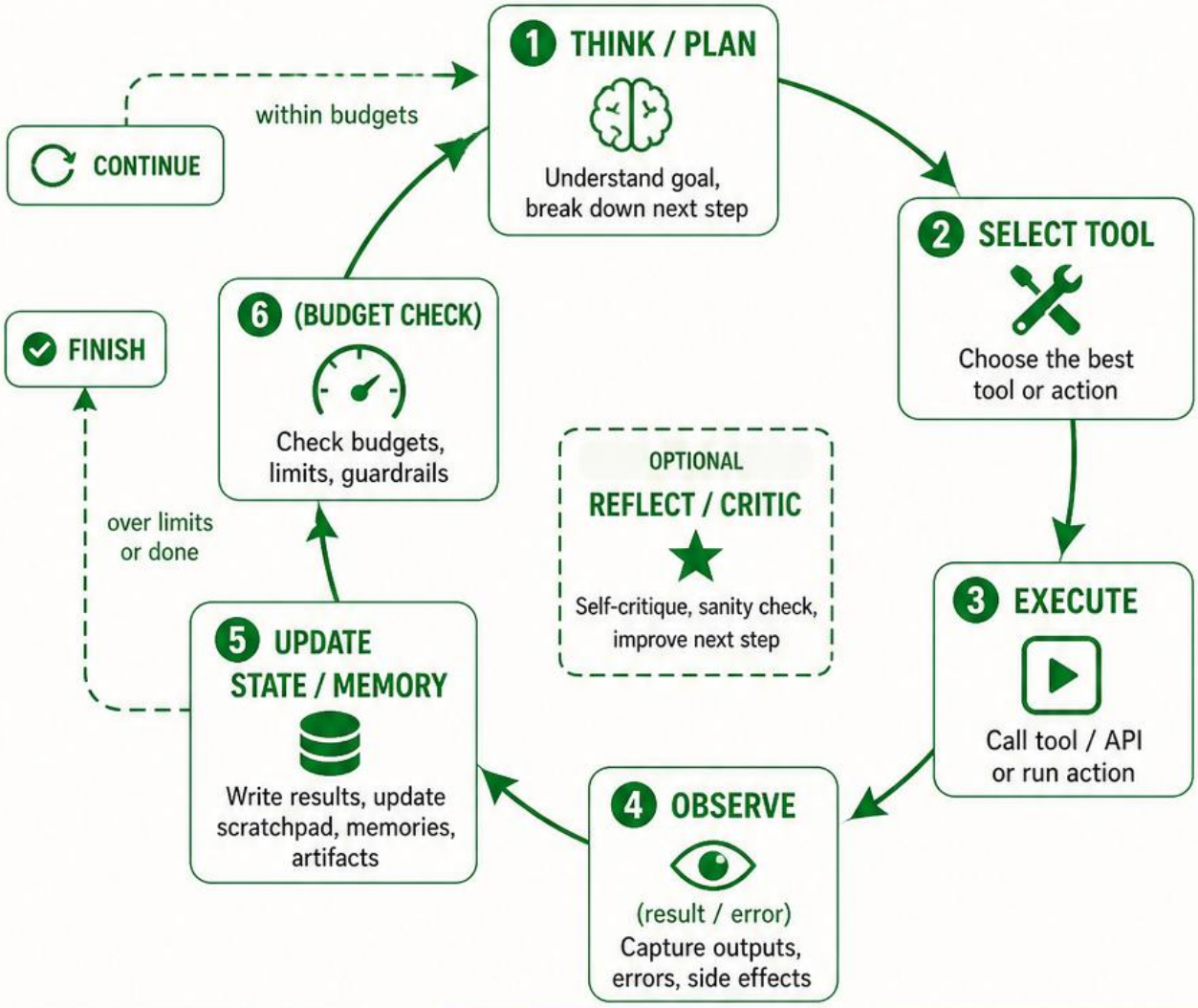
THE AGENT LOOP (AT A GLANCE)



☆ *Runtime owns the loop. Model proposes. Tools change the world.* ☆

03 — THE AGENT LOOP ★ KEY PAGE

Every serious agent is a state machine wrapped around an LLM call.



STATE

- goal
- plan scratchpad
- messages
- tool results
- artifacts
- step_count
- token_budget
- status

STOP CONDITIONS

- goal satisfied
- max steps
- max cost
- user cancel
- guardrail block
- unrecoverable error

BUDGETS

- max tool calls
- wall clock
- $ tokens

Hard limits in runtime not prompt.

⚠ **Never let the model alone decide infinite retries.**

★ **Hard budgets. Explicit stops. Durable step log.** ★

04 — TOOLS & FUNCTION CALLING

Tools are the agent's hands — typed, allowlisted, and side-effect aware.

TOOL SPEC

- **Name**
  Short, unique tool name.
- **Description**
  What it does, when to use it.
- **JSON schema (args)**
  Typed parameters and validations.
- **Auth scope**
  Required permissions / scopes.
- **Idempotent?**
  Safe to retry without duplication.
- **Timeout**
  Max execution time.
- **Rate limit**
  Max calls per user/IP/window.

Example (JSON Schema)

```
{
  "type": "object",
  "properties": {
    "query": {
      "type": "string",
      "minLength": 1
    },
    "limit": {
      "type": "integer",
      "minimum": 1,
      "maximum": 50
    }
  },
  "required": ["query"],
  "additionalProperties": false
}
```



CATEGORIES

READ-ONLY
Search, get, list data.
No side effects.

WRITE
Create, update, delete.
May change state.

CODE EXEC SANDBOX
Run code in isolated environments.

BROWSER
Navigate, scrape, interact with web pages.

RETRIEVAL
Vector search, RAG, document lookup.

DESIGN RULES

- **Least privilege**
  Give only the minimum scope required.
- **Confirm high-risk writes**
  Add human or policy confirmation for destructive actions.
- **Idempotency keys**
  Support retries safely; dedupe by key.
- **Structured errors to model**
  Return machine-readable errors with codes and messages.
- **Never expose secrets in tool output**
  Redact tokens, keys, PII, and internal URLs.

PITFALLS

- **Too many tools confuse routing**
  Keep the surface small and well-grouped.
- **Vague descriptions**
  Leads to wrong tool selection.
  Be specific and example-rich.
- **Giant blobs in context**
  Large outputs waste tokens and reduce accuracy. Summarize before returning.

Schema in. Validate. Execute. Summarize result back.

05 — PLANNING PATTERNS

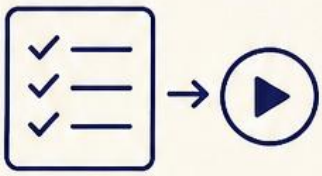
How the agent decides the next action — the control policy.

1 ReAct



interleave Thought / Action / Observation in one loop; good default

2 Plan-and-Execute



make full plan first, then execute steps; replan on failure

3 Router / Intent



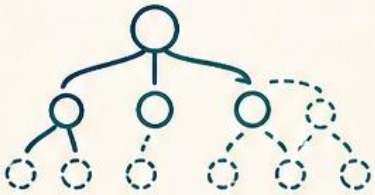
classify to specialist skill or tool subgraph

4 Reflection / Self-critique



draft → critique → revise before acting

5 Tree-of-Thoughts light



explore branches for hard search (costly)

WHEN TO PICK EACH

1	ReAct	Unclear path, tool-heavy, changing info, interactive tasks
2	Plan-and-Execute	Long, multi-step jobs, stable requirements, batch workflows
3	Router / Intent	Many skills/tools, domain routing, clear expertise boundaries
4	Reflection / Self-critique	Quality-sensitive tasks, writing, code, safety-critical actions
5	Tree-of-Thoughts light	Hard reasoning/search, puzzles, exploration problems

COST / LATENCY TRADEOFFS

★☆☆☆☆	Low cost, low latency
★★★☆☆	Higher cost, higher latency
★☆☆☆☆	Low cost, low latency
★★☆☆☆	Moderate cost, moderate latency
★★★★☆	High cost, high latency

★ Start ReAct; graduate to plan-execute for long jobs. ★

06 — Memory Systems

Context window is RAM. You still need disk — and eviction policy.

Memory Layers



L1	WORKING MEMORY	current messages + scratchpad (in prompt)
L2	EPISODIC	past runs / session summaries
L3	SEMANTIC	vector RAG facts, docs, user prefs
L4	ENTITY / PROFILE STORE	structured user/ ong state

Operations

WRITE
store new info at the right layer

RETRIEVE
find relevant info on demand

SUMMARIZE
compress long history

FORGET
evict or expire what's not useful

Strategies

SLIDING WINDOW
keep the most recent turns in context

SUMMARIZE OLD TURNS
roll up older content into summaries

PIN CRITICAL FACTS
keep must-know items in memory

RETRIEVE TOP-K
by embedding + filters (time, source, tags)

Pitfalls

STUFFING RAW TOOL DUMPS
bloats context and lowers quality

MEMORY POISONING
bad or injected info persists and misleads

STALE FACTS WITHOUT TIMESTAMPS
old info treated as current causes wrong decisions

☆ *Summarize the past. Retrieve the relevant. Pin the critical.* ☆

07 — STATE, WORKFLOWS & DURABILITY ★ KEY PAGE

Production agents are durable workflows — crash mid-tool must not double-charge.

STATE MACHINE

- queued
- running
- waiting_tool
- waiting_human
- succeeded / failed / cancelled

Persist after every step: step_id, inputs, outputs, idempotency key

DURABLE EXECUTION

- checkpoint
- resume
- exactly-once effects via idempotent tools
- Human approval as a blocking state

ORCHESTRATION STYLES

- code-first loops
- graph workflows (nodes/edges)
- event-driven sagas

Prompt-only agents die on process restart. Persist state.

★ **Checkpoint every step. Idempotent side effects.** ★

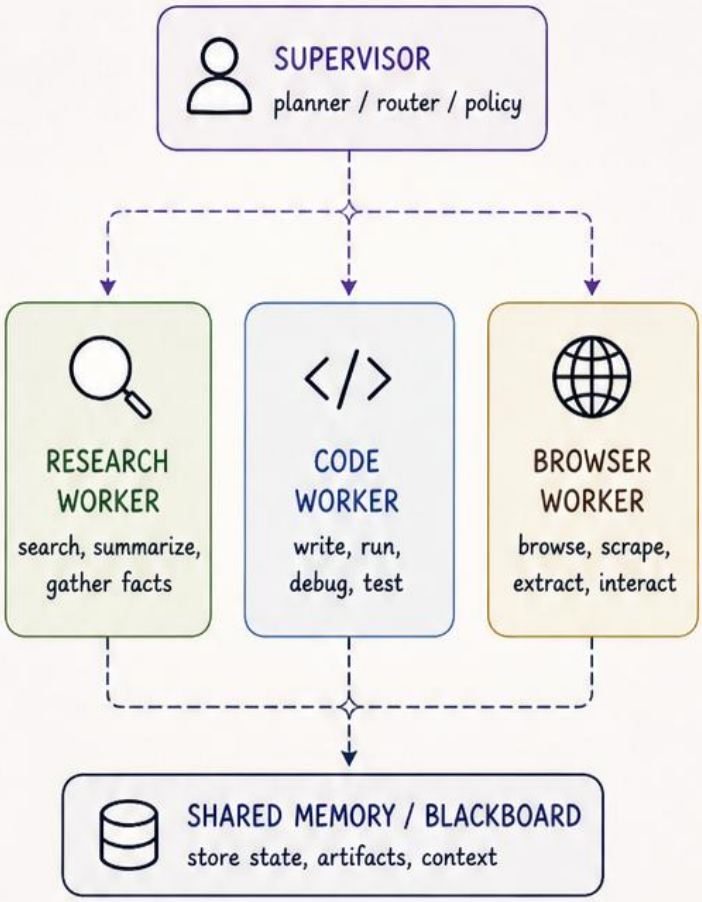
08 — Multi-Agent Systems

Split roles when one context and one prompt become a mess.

PATTERNS

1. **SUPERVISOR / WORKER**
   router assigns subagents
2. **PIPELINE**
   researcher → writer → reviewer sequential
3. **DEBATE / CRITIC**
   generator vs critic
4. **PARALLEL SPECIALISTS**
   merge results
5. **HANDOFF PROTOCOL**
   shared blackboard / message bus

SUPERVISOR PATTERN (EXAMPLE)



TRADEOFFS

- **More agents ≠ smarter**
  returns diminish.
- **Coordination tax**
  planning, routing, sync cost time.
- **Shared memory races**
  conflicts, stale reads, overwrites.
- **Cost multiplies**
  tokens, tools, latency add up.

WHEN TO USE

- **HETEROGENEOUS TOOLS / SKILLS**
  different capabilities, one goal.
- **LONG WORKFLOWS**
  many steps, many decisions.
- **NEED REVIEW GATE**
  quality, safety, compliance.

★ Specialize agents. Centralize policy and budgets. ★

09 — ORCHESTRATION LAYER

Frameworks differ; the primitives do not.

PRIMITIVES CHECKLIST

- [x] Prompt templates
- [x] Tool registry
- [x] Memory adapters
- [x] Graph/state
- [x] Tracing
- [x] Eval hooks
- [x] Human gates

MODEL ROUTING

Cheap model
for classify / route.

Strong model
for hard plan / code.

Embeddings model
for retrieve.

BUILD vs BUY

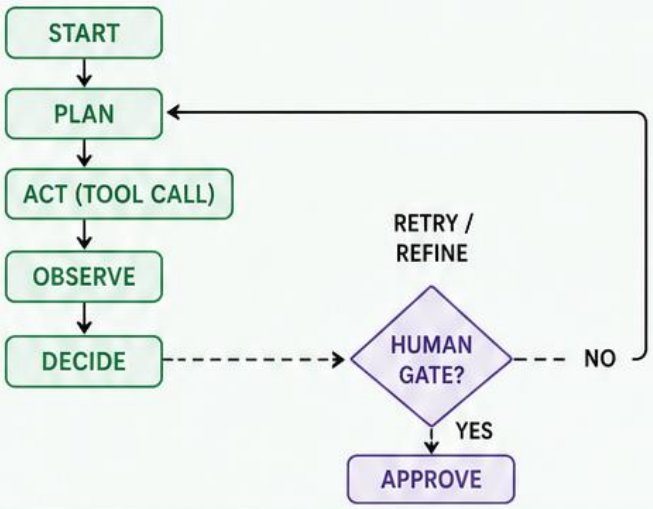
Start thin
Custom loop.

→

Adopt framework
When graphs, retries, and memory grow.

PROMPT STACK

1. System policy
2. Tools docs
3. Retrieved context
4. User goal
5. Scratchpad

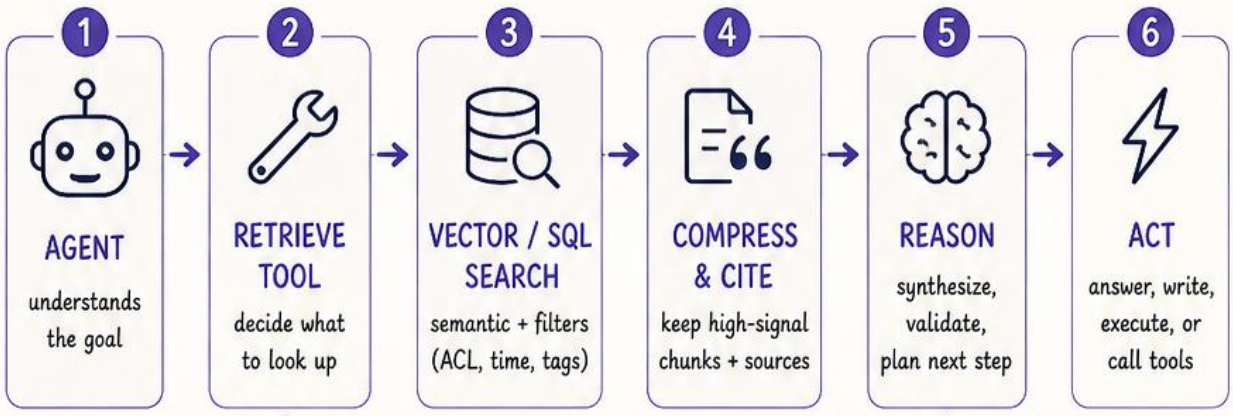


Primitives over brand names in the interview.

10 — RAG Meets Agents

RAG grounds answers. Agents decide when and what to retrieve — and what to do next.

AGENTIC RAG WORKFLOW (ITERATIVE)

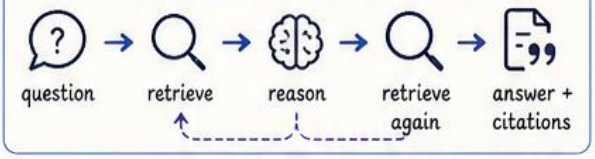


RAG PATTERNS

SINGLE-SHOT RAG



AGENTIC RETRIEVAL (MULTI-HOP)



AGENT TOOLS & TRICKS

QUERY REWRITE
rephrase, expand, and add filters for better recall

HYDE (LIGHT)
generate a hypothetical answer to improve retrieval

CITATION REQUIRED
must include sources before acting on docs

PITFALLS

RETRIEVED JUNK → TOOL MISUSE
bad filters or vague queries bring noisy/irrelevant docs

STUFFING
overloading context window hurts reasoning and quality

IGNORING TIMESTAMPS / ACL ON DOCS
stale or unauthorized info leads to wrong or unsafe acts

★ Retrieve with ACL. Cite before you act on knowledge. ★

11 — EVALUATION

★ KEY PAGE

If you cannot score the agent, you cannot ship the agent.

Unit	• tool schema, parsers, deterministic skills
Trajectory	• did it call the right tools in order?
Outcome	• task success / gold final state
Rubric	• LLM-as-judge carefully calibrated
Online	• thumbs, escalation rate, human edit distance

METRICS

- task success rate
- steps
- cost
- latency
- tool error rate
- hallucination / citation precision
- safety violations = 0

DATASETS

- golden tasks with expected tool traces
- regression suite on every prompt / model change

★ **Golden trajectories + outcome checks beat vibes.** ★

12 — SAFETY & GUARDRAILS

The model is persuasive. The runtime must be authoritative.

DEFENSE IN DEPTH

1. **Policy system prompt**
   Necessary, not sufficient.
2. **Input filters** — jailbreak/PII
3. **Tool allowlist + arg validation + dry-run**
4. **Output filters** — toxic/secret leak
5. **Human approval** for irreversible actions
6. **Sandbox** for code/browser
7. **Audit log** immutable

THREATS

Prompt injection via tool results/docs



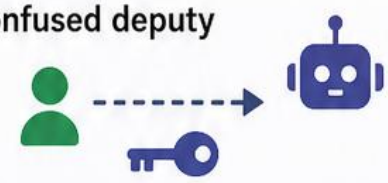
Data exfiltration



Runaway spend



Confused deputy



Treat retrieved text and web pages as untrusted.

Allowlist tools. Approve writes. Untrust content.

13 — RELIABILITY & FAILURE MODES

WHAT BREAKS	WHAT WE DO
Tool timeout	retry idempotent / fail soft
Tool wrong result	verify step / secondary check
Model loops	step budget / detect repeat actions
Context overflow	summarize / drop tool blobs
Partial write	compensate / saga
Provider outage	fallback model / queue
Prompt injection	strip/ignore tool-instruction patterns

DEGRADATION

shrink autonomy → suggest plan only → human takes over

Detect loops. Cap retries. Compensate writes.

14 — OBSERVABILITY & TRACING

★ An agent without traces is a black box you cannot debug or audit.

TRACE TREE (OpenTelemetry-style)

run_id	run_01JZX7K8Y3N6Z9Q5M9P7			8.42s
steps			5 steps	8.42s
LLM spans (prompt/completion tokens)			3 spans	3.21s
llm.step_1	gpt-4o	p: 512	c: 768	1.21s
llm.step_2	gpt-4o	p: 650	c: 512	1.02s
llm.step_4	gpt-4o-mini	p: 320	c: 256	0.98s
tool spans (args/latency/status)			4 spans	3.11s
tool.search_docs	{ "query": "risk factors" }		812ms	OK
tool.get_customer	{ "customer_id": "c_123" }		642ms	OK
tool.update_ticket	{ "ticket_id": "t_987" }		1.21s	OK
tool.send_email	{ "to": "user@example.com" }		438ms	ERR
retrieval spans (index/latency/hits)			2 spans	2.10s
retrieval.vector_search	index: docs_v3		1.24s	hits: 7
retrieval.rerank	model: bge-reranker-v2		856ms	hits: 5

All spans include trace_id, span_id, parent_id, start_time, end_time, attributes, events.

PRACTICES Redact secrets · Mask PII · Limit payloads · Set retention & TTL

PRINCIPLES

- Instrument every step and tool call.
- Correlate with run_id / trace_id.
- Capture tokens, latency, cost.
- Record status and errors.
- Follow OpenTelemetry conventions.
- Keep it structured and queryable.

WHAT TO CAPTURE

- **LLM:** model, prompt_tokens, completion_tokens, latency, cost
- **Tools:** name, args (redacted), latency, status, error
- **Retrieval:** index, query, hits, latency, reranker
- **Metadata:** user_id, session_id, prompt_version, model_version, tool_version, env

REDACTION EXAMPLES

SENSITIVE	REDACTED
Authorization: Bearer sk-...	Authorization: Bearer ***
"api_key": "sk-live-12345"	"api_key": "***"
"email": "alice@acme.com"	"email": "a***@acme.com"
"ssn": "123-45-6789"	"ssn": "***-**-6789"
"card": "4111 1111 1111 1111"	"card": "4111 **** **** 1111

Redact at source. Never log secrets.

DASHBOARDS (What to watch)



REPLAY & REPRODUCIBILITY

REPLAY

Store enough to reproduce a run:

- ✓ Original user input
- ✓ Resolved prompt versions
- ✓ Model name + version
- ✓ Tool name + version
- ✓ Tool args (redacted)
- ✓ Retrieval queries + index snapshot refs
- ✓ Random seeds / config
- ✓ Environment info



Determinism Goal
Same inputs + same versions = same (or explainably close) outputs & metrics

VERSION EVERYTHING

- ✓ Prompts → v1.3.0
- ✓ Models → gpt-4o@2024-05-13
- ✓ Tools → search_docs@2.1.0
- ✓ Indexes → docs_v3@2024-05-10
- ✓ Pipelines → agent_orchestrator@1.7.2
- ✓ Evaluations → evals_suite@3.0.1

Store lineage. Make rollbacks easy.

☆ Trace every step. Redact secrets. Version everything.

15 — COST, LATENCY & MODEL ROUTING

Agents multiply tokens — budget is an architecture constraint.

COST DRIVERS

- [x] Input context size
- [x] Steps
- [x] Tool result echo
- [x] Multi-agent fan-out
- [x] Retries

LATENCY DRIVERS

- [x] Serial tool calls
- [x] TTFT (time to first token)
- [x] Human waits

TACTICS TO CONTROL COST & LATENCY

- [x] Smaller model for route / extract
- [x] Cache prompts / RAG
- [x] Summarize tool output
- [x] Parallelize independent tools
- [x] Early exit
- [x] Stream UX

NORTH-STAR METRIC

Cost per successful task.
(Not cost per call.)

Route cheap. Parallelize safe. Cap steps.

16 – Production Reference Stack

1. CLIENTS / CHANNELS

Web · Mobile · Chat / UI · Webhooks · Enterprise Apps

2. API GATEWAY + AUTH

AuthN / AuthZ (OIDC, OAuth2) · Rate Limits & Quotas · API Keys & Scopes · Request Validation

3. AGENT RUNTIME (LOOP, STATE, BUDGETS)

Loop Orchestrator · State & Checkpoints · Budgets & Guardrails · Stop / Escalate Conditions

4. MODEL GATEWAY (PROVIDERS, ROUTING, FALLBACK)

Routing & Policy · Provider Adapters · Fallback & Retry · Cost / Latency Aware

5. TOOL MESH (CONNECTORS, SANDBOX, SECRETS)

Tool Connectors · Sandbox & Isolation · Secrets Management · Allowlist / Policy Enforcement

6. MEMORY & RAG (VECTOR + SQL + KV)

Vector Store (Embeddings) · SQL Store (Relational) · KV Store (Cache) · Rerank & Context

7. WORKFLOW DB / QUEUE

Workflow DB (Runs & State) · Message Queue (Async Tasks) · Scheduler (Cron / Delayed) · Idempotency & Locks

8. OBSERVABILITY + EVAL + POLICY CENTER

Tracing & Logs · Metrics & Dashboards · Evaluations & Benchmarks · Policy Center (Guardrails) · Audit & Compliance

9. HUMAN OPS CONSOLE

Run Monitor & Timeline · Escalations & Approvals · Override & Notes · Safe Actions & Playbooks

DEPLOY

Queue long runs

Use workflow DB + queue for reliability.

Webhooks for async tools

External tools respond via webhooks.

Multi-tenant isolation

Tenant-aware keys, data isolation, and per-tenant quotas.

☆ *Gateway the models. Mesh the tools. Persist the runs.* ☆

17 — AGENT DESIGN PATTERNS

1 Tool-use assistant

WHEN: You need the agent to use one or more tools/APIs to get work done.

2 ReAct worker

WHEN: Tasks need interleaved reasoning and actions with feedback from the environment.

3 Plan-execute project manager

WHEN: Complex, multi-step goals that benefit from planning, tracking, and progress updates.

4 RAG researcher

WHEN: Answers require up-to-date or domain-specific knowledge from external documents.

5 Coding agent (edit-run-test loop)

WHEN: Writing or modifying code that must be executed and verified iteratively.

6 SQL analyst (schema→query→verify)

WHEN: Data questions that need schema understanding, query generation, and result validation.

7 Browser operator (observe DOM→act)

WHEN: Web tasks like extraction, form filling, navigation, or automation via the UI.

8 Supervisor multi-agent

WHEN: Orchestrating multiple specialist agents to complete a broader objective.

9 Human-approval gate

WHEN: High-risk or irreversible actions that must be reviewed and approved by a human.

10 Evaluator-optimizer pair

WHEN: Quality, accuracy, or performance needs iterative evaluation and improvement.

Pick a pattern; don't invent chaos.

18 — INTERVIEW Q & A

Q Agent vs chatbot?

A Agent = loop that plans, uses tools, keeps state, and stops when the goal is met.

Chatbot = one-shot answer from model memory.



Q How prevent infinite loops?

A
- Max steps / max cost per run.
- Detect repeat actions or no progress.
- Hard runtime caps & budget guards.



Q How prevent double side effects?

A Use idempotency keys for every write/charge and checkpoint before/after tool calls.



Q Prompt injection via docs?

A
- Treat retrieved content as untrusted.
- Tool allowlist only.
- Never obey instructions in retrieved text.
- Apply output filters & safety policies.



Q When use multi-agent?

A When tasks need clear role separation, parallel work, or independent review.

Otherwise, prefer a single agent with tools.



Q How do you eval agents?

A Run golden tasks with expected outcomes.
Score tool call traces + final results.
Track regressions across releases.



✕ COMMON MISTAKES TO AVOID

No budgets



Unbounded runs lead to runaway cost and latency.

All-powerful tools



Over-permissioned tools increase blast radius.

No traces



You can't debug or improve what you don't observe.

Framework name-dropping without primitives



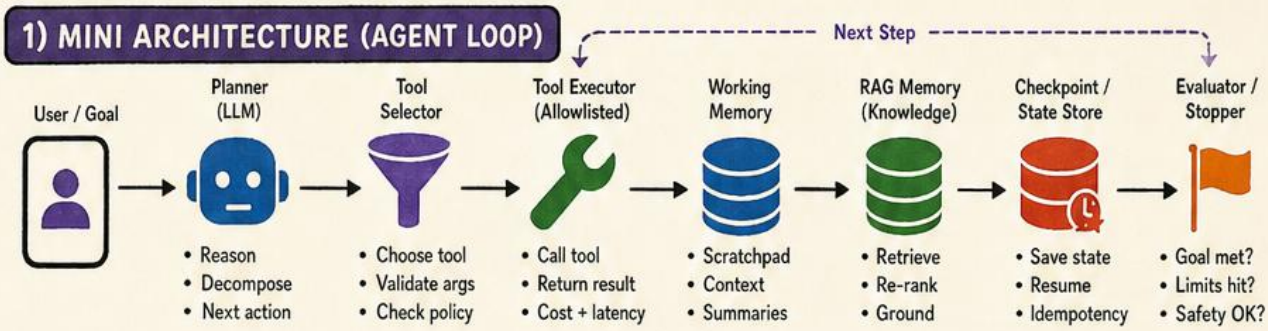
Buzzwords ≠ systems. Understand the core primitives.

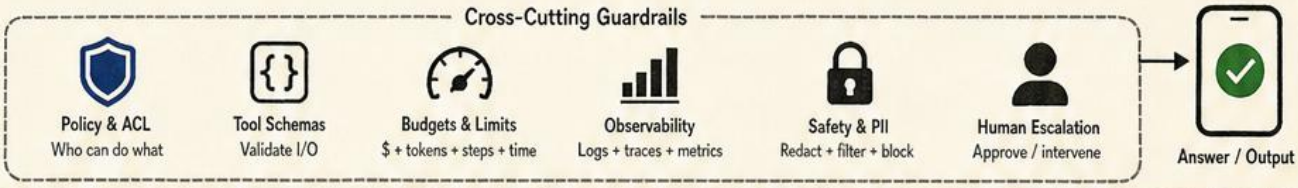
★ Goal, tools, budgets, eval — say these four.

19 — RECAP & CHEAT SHEET

Wrap an LLM in a **budgeted loop** that plans, calls **allowlisted tools**, remembers via **working+RAG memory**, **checkpoints state**, and **stops** when the goal or limits hit.

1) MINI ARCHITECTURE (AGENT LOOP)





2) TOP 10 TALKING POINTS

1. Agent = LLM + Tools + Memory + Guardrails.
2. Plan → Select tool → Execute → Observe → Repeat.
3. Tools are allowlisted with strict schemas.
4. Validate arguments & outputs at every step.
5. Working memory holds short-term scratchpad.
6. RAG memory grounds answers in trusted data.
7. Budget everything: $ spend, tokens, steps, time.
8. Stop when goal met or any limit/safety trip.
10. Sbg; measure, alert, and keep humans in the loop.

3) NUMBERS MINDSET

$ / TASK (BUDGET)	Set a hard cap. Track actual vs budget. Alert at 80–90%.
P95 LATENCY	Keep p95 low. Stream early. Cache & parallelize.
SUCCESS RATE	Define success clearly. Measure end-to-end. Improve with evals.

4) AGENT LOOP (STEPS)

1. Ingest goal + context + constraints.
2. Plan: break down & decide next action.
3. Select best allowlisted tool.
4. Validate args against schema & policy.
5. Execute tool (observe cost & latency).
6. Update working memory with results.
7. Retrieve from RAG memory if needed.
8. Evaluate: goal met or limits hit?
9. Checkpoint state & either stop or loop.

5) TOOLING PRINCIPLES

- ✓ Small, focused tools with one job.
- ✓ Explicit schemas (input/output/examples).
- ✓ Deterministic when possible.
- ✓ Idempotent & safe by default.
- ✓ Least-privilege credentials.
- ✓ Rate limits, retries with backoff.
- ✓ Clear error types & messages.
- ✓ Version tools, renew break contracts.

6) MEMORY DESIGN

WORKING MEMORY (SHORT-TERM)
- Scratchpad, recent messages, tool results.
- Summarize to control context size.
- Keep only what's needed now.

RAG MEMORY (LONG-TERM)
- Selective ingest of trusted data.
- Embed → index → retrieve → re-rank.
- Cite sources; prefer grounded answers.

CHECKPOINT / STATE
- Save after each step (state, cost, phase).
- Resume after failures & timeouts.
- Deduplicate with idempotency keys.

7) SAFETY & GOVERNANCE

- ✓ Enforce ACLs on every tool call.
- ✓ Mask/strip PII and secrets.
- ✓ Content filters & policy checks.
- ✓ Prompt injection detection.
- ✓ Human-in-the-loop for risky actions.
- ✓ Audit logs (who, what, when, why).
- ✓ Data retention & deletion policies.
- ✓ Compliance: SOC2, GDPR, etc.

8) OBSERVABILITY & EVALS

LOGS	Every step, tool call, args, result, error.
TRACES	End-to-end spans, latency breakdown.
METRICS	Cost, tokens, latency (p50/p95), steps.
EVALS	Offline test sets + online success rate.

Dashboards • Alerts • Anomaly detection

9) COMMON FAILURE MODES & FIXES

✗ Tool misuse	Tight schemas, examples, validation, retries.
✗ Context bloat	Summarize, prune, retrieval, window mgmt.
✗ Hallucinations	RAG grounding, citations, confidence checks.
✗ Runaway loops	Step/time limits, stop rules, loop detector.
✗ High cost	Budgets, caching, cheaper models, batching.
✗ Privacy leaks	PII filters, encryption, strict access controls.

LOOP WITH BRAKES. TOOLS WITH SCHEMAS. MEMORY WITH ACL.